

ADR-04: Incorporación de API REST a ZenFlow

ZenFlow — Exponer tareas y hábitos mediante ASP.NET Core Web API con Swagger

Campo	Valor
Autor	Dylan Vazquez
Fecha	19/06/2026
Estado	Propuesto

Contexto

ZenFlow comenzó como una aplicación de escritorio local donde todo — UI, lógica y datos — vivía en la misma máquina del estudiante. Esa arquitectura resolvió el problema inicial, pero tiene un límite claro: los datos de ZenFlow son inaccesibles desde afuera. Si alguien quisiera consultar sus tareas desde un navegador, una app móvil o cualquier otra herramienta, no habría forma de hacerlo.

La incorporación de una API REST resuelve ese problema sin romper lo que ya existe. Al exponer las operaciones de tareas y hábitos a través de endpoints HTTP estándar, ZenFlow pasa de ser una app cerrada a ser un sistema con una capa de acceso abierta y documentada.

Las condiciones que influyeron en esta decisión:

- La lógica de negocio ya está encapsulada en GestorTareas y GestorHabitos, y los repositorios ya están abstraídos detrás de interfaces. Agregar una API no requiere reescribir nada — solo crear un nuevo proyecto que consuma esas mismas clases.
- ASP.NET Core es parte del ecosistema .NET 8 que ya uso en el proyecto WPF. No hay que aprender un framework nuevo.
- Swagger (OpenAPI) es el estándar de la industria para documentar APIs REST. Configurarlos en ASP.NET Core toma menos de diez líneas y genera documentación interactiva automáticamente.
- El proyecto vive en la misma solución de Visual Studio, lo que significa que la API y la app WPF comparten los mismos modelos y lógica sin duplicar código.

Decisión

Agregar un proyecto ASP.NET Core Web API llamado ZenFlow.API dentro de la misma solución de Visual Studio, que exponga los endpoints de tareas y hábitos usando los mismos GestorTareas y GestorHabitos que ya usa la app WPF, con Swagger configurado para documentación interactiva.

Los endpoints implementados son:

Método	Endpoint	Descripción
GET	/api/tareas	Obtiene todas las tareas pendientes
POST	/api/tareas	Crea una nueva tarea
DELETE	/api/tareas/{id}	Elimina una tarea por Id
GET	/api/habitos	Obtiene todos los hábitos
POST	/api/habitos	Crea un nuevo hábito
PUT	/api/habitos/{id}/cumplir	Registra cumplimiento del día
DELETE	/api/habitos/{id}	Elimina un hábito por Id

¿Por qué REST y no otra alternativa?

REST se eligió porque es el estilo arquitectónico más universal para APIs HTTP. Cualquier cliente — un navegador, Postman, una app móvil, otro servicio — puede consumirlo sin instalar nada especial. Los verbos HTTP (GET, POST, PUT, DELETE) mapean directamente a las operaciones que ZenFlow ya tiene en su lógica de negocio, lo que hace que la traducción sea natural y sin fricción.

La razón concreta por la que se eligió ASP.NET Core sobre otras opciones es que ya estamos en el ecosistema .NET 8. Crear un Web API project en Visual Studio toma segundos, y la inyección de dependencias nativa de ASP.NET Core permite pasar GestorTareas y GestorHabitos a los controladores exactamente igual que se pasan a cualquier otra clase — sin cambiar nada de la arquitectura existente.

Alternativas consideradas

Alternativa	Por qué la descarté
GraphQL	Resuelve un problema que ZenFlow no tiene: consultas complejas y anidadas donde el cliente elige exactamente qué campos recibe. Para exponer tareas y hábitos con operaciones simples (GET, POST, DELETE), esa flexibilidad es innecesaria y agrega complejidad en el servidor y en los clientes que consuman la API.
gRPC	Es más eficiente que REST para comunicación entre servicios internos, pero requiere que todos los clientes usen el mismo protocolo binario y generen código desde un archivo .proto. Si mañana alguien quiere consumir la API de ZenFlow desde un navegador o Postman, gRPC complica la integración sin ofrecer una ventaja real a esta escala.
SignalR (WebSockets)	Diseñado para comunicación en tiempo real — notificaciones push, chats, dashboards en vivo. ZenFlow no necesita eso: las tareas y hábitos se consultan y modifican por petición del usuario,

WCF (Windows Communication Foundation)	no en tiempo real. Sería sobreingeniería para el caso de uso actual. Tecnología de Microsoft anterior a REST. Está en modo mantenimiento, no se recomienda para proyectos nuevos en .NET 8 y tiene una configuración significativamente más compleja. No hay razón para usarla cuando ASP.NET Core Web API existe.
--	--

Consecuencias

Lo que gana:

- Técnica: ZenFlow deja de ser una app de escritorio cerrada y se convierte en un sistema con una capa de acceso pública. Cualquier cliente que hable HTTP puede consultar o modificar tareas y hábitos sin tocar el código de la app WPF.
- Técnica: gracias al patrón repositorio ya implementado, los controladores de la API usan exactamente las mismas interfaces (ITareaRepo, IHabitoRepo) que la app WPF. No hay duplicación de lógica — el mismo GestorTareas sirve a la app de escritorio y a la API simultáneamente.
- Proceso: Swagger genera documentación interactiva automáticamente. Cualquier persona que reciba la URL del servidor puede ver todos los endpoints, sus parámetros y probarlos directamente desde el navegador, sin necesidad de documentación externa.

Lo que sacrifico o asumo:

- Limitación técnica: la API y la app WPF comparten los mismos archivos JSON como fuente de datos. Si ambas corren al mismo tiempo y modifican los mismos archivos, puede haber condiciones de carrera — dos escrituras simultáneas que corrompan el JSON. Esto no es un problema mientras la API se use en desarrollo, pero sería un riesgo real en producción con múltiples usuarios concurrentes.
 - Deuda técnica: la API no tiene autenticación. Cualquiera que conozca la URL puede leer y modificar los datos. Para una herramienta personal de escritorio en desarrollo esto es aceptable, pero si la API se expone públicamente habría que agregar JWT u otro mecanismo de autenticación.
 - Deuda técnica: al migrar de JSON a SQLite en el futuro, la API hereda automáticamente ese cambio gracias al DIP — pero también hereda el trabajo de migrar los datos existentes.
-

Diagrama

